

React

Begriffe kurz

React vor allem für SPA, JS-Library

Komponente

Props alle JS Datentypen möglich, als destructured object oder als props Objekt annehmen. Props sind Eingabewerte für Komponenten, immutable, Parent zu Child

JSX Syntax für React Komponente, verbindet HTML und JS - wird übersetzt in JS, JavaScript XML

Expressions sind Werte oder Kombinationen von Werten, Variablen und Operatoren, die ein Resultat ergeben.

List Rendering Eine Anzahl gleicher Komponenten darstellen, aber mit unterschiedlichen

Inhalten `.map()`

Conditional Rendering

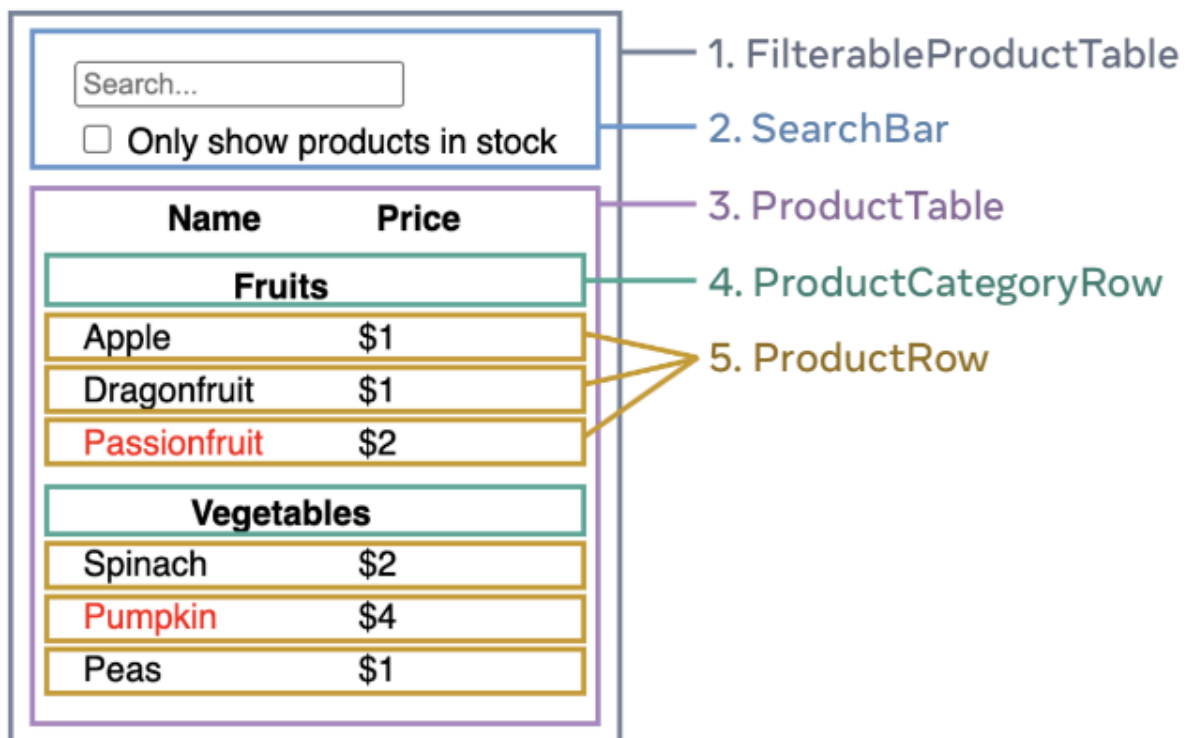
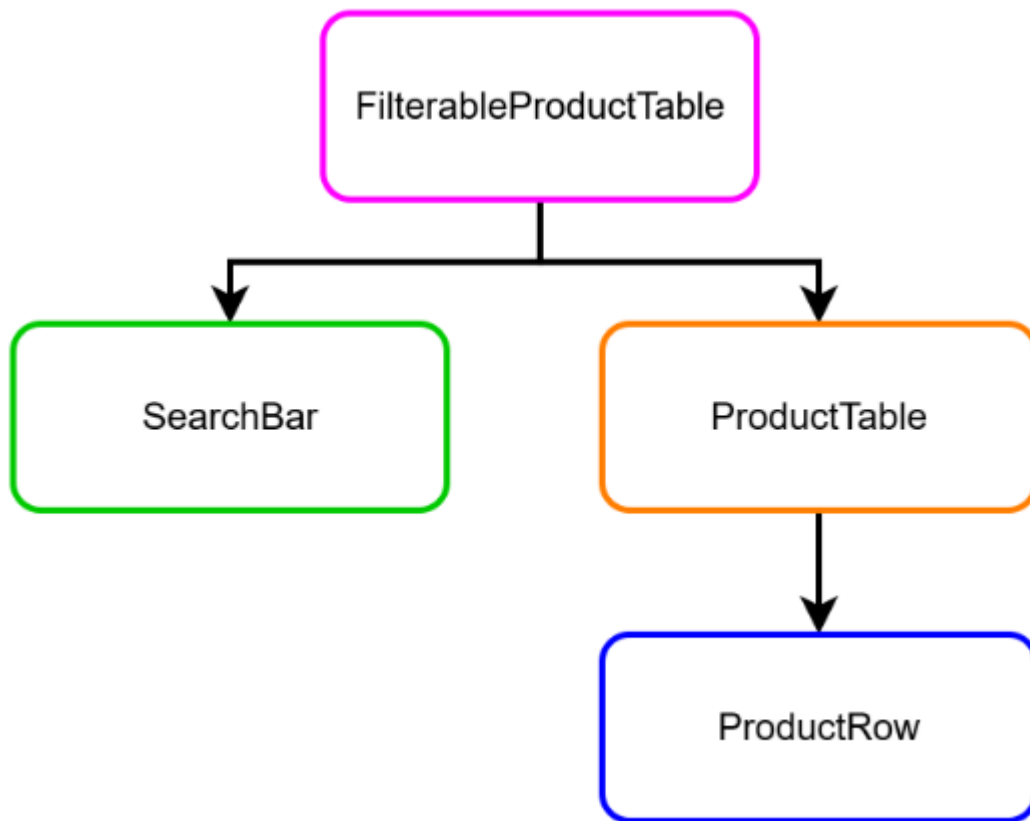
- `.filter()`

State Zustand einer Komponente

- Der State kann durch Interaktionen verändert werden z. B. Klickcounter, Suchtext den der Nutzer eingibt, Wert einer Checkbox
- bleibt nicht unverändert, nicht als props erhalten, kann man nicht aus anderen Daten berechnen

Wie baut man UIs in React? (Thinking in React) / Wie komme ich von einer Skizze/Wireframe/... zu einer Anwendung?

- 1) **Eine Idee haben, wie es aussehen soll (Skizze, Mockup, HTML/CSS)** Das kann ein Mockup (links) sein, ein (HTML-, Papier-, Figma-, ...)Prototyp, ein Datenmodell, eine JSON-API-Response
- 2) **Zerlegen in eine Komponentenhierarchie**



3) Eine statische Version in React implementieren Statisches UI in React

```

const FilterableProductTable = ({ products }) => {
  // ...
};
  
```

```

const SearchBar = () => {
  // ...
};
const ProductTable = ({ products }) => {
  // ...
};
const ProductRow = (props) => {
  // ...
};

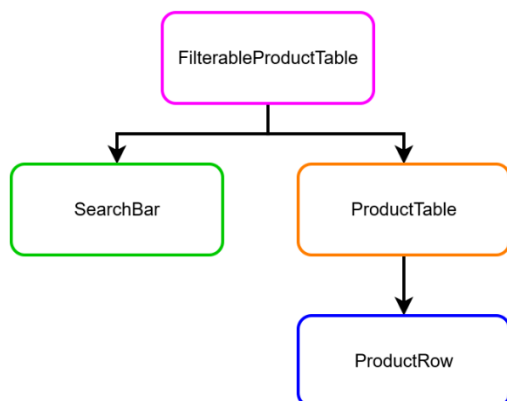
```

4a. **Den minimalen, kompletten UI Zustand (State) finden** Um die Anwendung interaktiv zu machen, müssen User das Datenmodell anpassen können. Wir verwenden dafür State. Der State (Zustand) der Anwendung sollte das kleinstmögliche Set von Daten sein, die sich ändern und die Anwendung sich merken soll.

Prinzip für die Struktur des State -> DRY (Don't repeat yourself) "Make your state as simple as it can be — but no simpler."

4b. **Herausfinden, wo der State leben soll** Jetzt kennen wir den State - Wo in der Anwendung kommt der hin? Grundsätzlich kann jede Komponente State besitzen. Wir erinnern uns, Daten können durch Props immer nur von Parent zu Child weitergegeben werden.

Vorgehen für jedes Stück Zustand: Alle Komponenten identifizieren, die etwas aufgrund dieses Stück Zustands darstellen. Den nächsten gemeinsamen Parent in der Hierarchie finden. Entscheiden, wo der Zustand leben soll Häufig ist es der **gemeinsame Parent** Manchmal ist es eine höhere Komponente als der gemeinsame Parent Wenn es keine Komponente gibt, wo die Platzierung des States Sinn ergibt, erstelle eine neue Komponente und platziere sie in der Hierarchie überhalb des gemeinsamen Parents



- Wo lebt der State?
 - Den Suchtext, den der Nutzer eingegeben hat
 - Brauchen wir in SearchBar und in (Filterable)ProductTable
 - -> FilterableProductTable
 - Den Wert der Checkbox
 - Brauchen wir in SearchBar und in (Filterable)ProductTable
 - -> FilterableProductTable

- Nutze Eventhandler

5) Datenflow von Child zu Parent aufbauen

React Regeln

- Nur ein Rückgabewert (1 parent element oder React.Fragment <></>)

Naming Komponente

- UpperCamelCase
- treffende Beschreibung der Komponente
 - Beispiele: LinkButton , InfoTooltip , DraftEditor

HTML

- HTML Attribut “class” -> “className” (weil ‘class’ in JS ein reserviertes Keyword ist) oder “for” in Formularfeldern -> “htmlFor”
 - HTML-Attribute mit Bindestrich werden zu **lowerCamelCase**, bsp. stroke-width -> strokeWidth
- Alle Elemente müssen geschlossen sein, auch wenn sie keine child-Elemente beinhalten (in React strenger als in HTML).
 - ausser aria-* und data-*
 - Jedes öffnende Element braucht ein Schliessendes:
 - Leere Elemente müssen mit einem schliessend Slash / ausgezeichnet werden: , <input /> oder auch leere React Komponenten <Article />
- Comments in JSX werden so geschrieben: { /* comment */ } im JavaScript Teil werden Comments regulär ausgezeichnet

JSX

- Nur ein einziges Root Element zurückgeben (-> Siehe React Fragment)
- HTML Attribut “class” -> “className” (weil ‘class’ in JS ein reserviertes Keyword ist) oder “for”

in Formularfeldern -> “htmlFor”

- HTML-Attribute mit Bindestrich werden zu lowerCamelCase, bsp. stroke-width ->

strokeWidth

- Alle Elemente müssen geschlossen sein, auch wenn sie keine child-Elemente beinhalten (in React strenger als in HTML).
- Jedes öffnende Element braucht ein Schliessendes:
- Leere Elemente müssen mit einem schliessend Slash / ausgezeichnet werden: , <input /> oder auch leere React Komponenten <Article />
- Comments in JSX werden so geschrieben: {} im JavaScript Teil werden Comments regulär ausgezeichnet

Vanilla JS

Destructuring an Array

```
const Point = [20,300]
const [x, y] = Point
console.log(x) // Output: 20
console.log(y) // Output: 300
```

EventHandler

Wir brauchen Funktionen, die Interaktionen im UI mit dem State verbinden -> EventHandler.

<button onClick={() => setCounter(counter+1)}>Click Me</button> Native HTML Elemente haben in React spezielle Attribute/Props, die EventHandler aufnehmen. Einige der häufig verwendeten: onClick für Buttons onChange für Inputfelder onFocus/onBlur Diesen Attributen können wir EventHandler-Funktionen übergeben. EventHandler-Funktionen bekommen ein Eventobjekt e, das analog zum Event in JS funktioniert und Informationen zum Event enthält

Es gibt verschiedene Möglichkeiten, EventHandler zu platzieren:

```
// separat -> für längere Handler, oder mehrfach verwendete (Demo 1)
const clickHandler = () => { alert('submitted') }
return (
  <button onClick={clickHandler}>Click Me</button>
)
```

```
// inline -> für kurze Handler
return ( <button onClick={() => alert("submitted")}>Click Me</button> )
```

Stolperfalle: Funktionen, die an Eventhandler Attribute übergeben werden, dürfen nicht aufgerufen werden. Der Eventhandler ruft die Funktion auf. Beispiel, wie nicht:

```
// Falsch
return ( <button onClick={clickHandler()}>Click Me</button> )
```

Eventhandlers als Props übergeben

Häufig, wie in der Demo, übergeben wir Eventhandler als Props an Komponenten, um State in höhergelegenen Komponenten zu aktualisieren.

```
<SearchInput
  value={searchText}
  onChange={(e) => setSearchText(e.target.value)}
/>
```

Üblicherweise werden diese Props analog der nativen Eventhandler-Props mit `onThingThatHappens` benannt, wie `onUpdateSearchText` oder `onCloseDialog`

Expressions

Was sind alles Expressions?

```
9 // 9
"under the bridge" // "under the bridge"
(5 + 6) * 10 // 110
const height = 180 // keine Expression, sondern ein Statement
height * 10 // 1800
`you are ${height} cm tall` // 'you are 180 cm tall'
"Red" + " " + "Roses" // 'Red Roses'
const isPacked = false // keine Expression, sondern ein Statement
isPacked && "Du hast das gepackt" // false
isPacked ? "Du hast das gepackt" : "Das fehlt noch" // 'Das fehlt noch'
const people = ['tom', 'betty', 'van gogh'] // keine Expression
people.length // 3
```

React rendering Fehler:

```
<p>{obj}</p> {/* Error "Objects are not valid as a React child" */}
<p>{fn}</p> {/* Error "Functions are not valid as a React child" */}
```

Arrays

immer keys dazuschreiben `key={..}`!

- Sobald die Liste umsortiert wird oder anderweitig dynamisch ist, braucht es einen Key, sonst

kann es Bugs geben

```
// Liste filtern mit .filter
const people = [
  {name: 'Creola Katherine Johnson', profession: 'mathematician'},
  {name: 'Percy Lavon Julian', profession: 'chemist'},
];
const chemists = people.filter(person => person.profession === 'chemist');

// .map
<ul>{people.map(person => <li key={person}>{person}</li>)}</ul>

function MyCars() {
  const cars = [
```

```

{id: 1001, brand: 'Ford'},
{id: 1002, brand: 'BMW'},
{id: 1003, brand: 'Audi'}
];
return (
  {cars.map((car) => <li key={car.id}>I am a { car.brand }</li>)}
);
}

```

Operators

? : Ternary Operator (MDN) - Expression-Variante von if & else

```

return <li className="item"> {name} {isInStock ? '✓' : 'x'} </li>;
isInStock ? name + '✓' : name

```

&&Logischer AND Operator (MDN) - Expression-Variante von if

- Wenn die Bedingung falsch (false) ist, sieht React das als kein Inhalt an, wie auch null
- && gibt ersten falschen Wert zurück oder letzter Wert wenn alle true. Bei React/JSX wir false einfach nicht gerenderet.

```

return <li className="item">{name} {isInStock && ✓}</li>;
isInStock &&
//reads as
//if isInStock is true, then (&&) render the checkmark, otherwise, render nothing
oder undefined

```

```

result = 0 && 2; // result is assigned 0 -> Falsy.
// Aber react rendert ein null, andere falsy werte nicht
result = "text" && ""; // result is assigned "" -> falsy.
// (empty string -> falsy)
result = "piep" && 4; // result is assigned 4 -> truthy

```

Ein false Wert in JSX wird einfach ignoriert! Genauso wie true, null. Deshalb kann man mit && wenn etwas wahr ist, einen Wert ausgeben.

```

function Paragraph({text}) {
  return <p>{false}{true}{null} {text} {isPacked && "gepackt"}</p>;
}

```

—

```

// Do
{item.isInStock && <ListItem item={item}>}
// Don't
if (isInStock) {
  return null;
}
return <li className="item">{name}</li>;

```